

我会按“先数学本质，再工程本质”的顺序讲：量化为什么能省显存和算力，算子优化到底优化什么，以及为什么同一个算法在不同硬件上写法会完全不同。

这里有一个核心区分：**量化主要是在改变“数值表示”，而算子优化主要是在改变“同一个数学计算在硬件上的执行方式”**。后面我会把这两个层次分开讲，避免混在一起。

已思考 1m 19s >

下面这句话先抓住核心：

大模型量化的本质，是把连续高精度数值投影到低比特离散格点上，用可控误差换显存、带宽和低精度计算吞吐；算子优化的本质，是在不改变数学语义的前提下，把同一个计算重排成更适合硬件执行的数据流和指令流。

1. 大模型量化的本质

大模型本来大量使用 FP16 / BF16 / FP32 存权重和激活。比如一个线性层：

$$y = xW^T + b$$

其中 W 是权重， x 是输入激活。量化要做的事不是“把模型变小”这么简单，而是把 W 、 x 、KV cache 等张量从高精度数值变成低比特表示，例如 INT8、INT4、FP8、FP4。

更本质地说，量化是在做一个近似：

$$f_{\theta}(x) \approx f_{\hat{\theta}}(x)$$

其中原模型参数是 θ ，量化后参数是 $\hat{\theta} = Q(\theta)$ 。目标是：

$$\min_Q \mathbb{E}_{x \sim \mathcal{D}} [\mathcal{L}(f_{\theta}(x), f_{Q(\theta)}(x))]$$

也就是说，**不是追求每个权重都一模一样，而是追求模型整体输出尽量不变。**

2. 量化的数学机理：连续值 $\rightarrow$ 离散格点

最常见的是线性 / affine 量化。给一个高精度数值 x ，量化成整数 q ：

$$q = \text{clip} \left(\text{round} \left(\frac{x}{s} + z \right), q_{\min}, q_{\max} \right)$$

反量化为：

$$\hat{x} = s(q - z)$$

其中：

s : scale · 缩放因子
 z : zero point · 零点
 $q_{\min}, q_{\max}$: 低比特整数可表示范围



比如 INT8 的范围通常是 $[-128, 127]$ 。TensorRT 文档里对 INT8 量化也采用类似的“除以 scale、round、clip，再用 scale 反量化”的形式；PyTorch torchao 文档也说明，大多数张量量化使用 affine mapping，并且低精度张量通常还需要 scale、zero point、packing format 等元数据。 [👁 NVIDIA Docs +1](#)

量化误差来自两个地方：

$$e = x - \hat{x}$$

如果 x 没有被 clip，只是落到最近的量化格点，那么误差大约受限于：

$$|e| \leq \frac{s}{2}$$

但如果 x 超出量化范围，会发生 clipping：

原始值：	-10	-2	-1	0	1	2	10
量化可表示：		-2	-1	0	1	2	
超出范围：	被截断						被截断



所以量化的核心矛盾是：

scale 太大：大数不容易溢出，但小数分辨率很差。

scale 太小：小数表示更细，但大数容易被 clip。

3. 为什么大模型量化特别难？

因为大模型权重和激活不是均匀分布的，尤其 Transformer 里经常有 **outlier channel / outlier activation**。也就是某些通道的数值特别大。

假设一个通道里大多数值在 $[-1, 1]$ ，但有一个值是 20。如果你用整个 tensor 的最大值决定 scale，那么：

$$s = \frac{20}{127}$$

这会导致 $[-1, 1]$ 里的大量普通值被映射到很粗的整数格点上，误差变大。

所以大模型量化通常不会只用 per-tensor scale，而会用：

per-tensor quantization : 整个张量一个 scale
per-channel quantization : 每个输出通道一个 scale
per-group quantization : 每组若干权重一个 scale
block quantization : 每个 block 一个 scale



粒度越细，误差通常越小，但代价是 scale 更多、访存更复杂、kernel 更难写。TensorRT 文档也提到 dynamic block quantization 会按 block 动态计算 scale，因为 block 的动态范围通常小于整个 tensor，所以可以降低量化误差。 [NVIDIA Docs](#)

LLM.int8 的思路也体现了这个问题：普通特征用 INT8 矩阵乘，异常 outlier 维度单独用高精度处理，从而避免少数异常值毁掉整体量化精度。论文描述它用 vector-wise quantization 处理大多数特征，同时把 outlier feature dimensions 隔离到 16-bit 矩阵乘里。 [arXiv](#)

4. 在线性层里，量化到底怎么省算力？

原始线性层：

$$y_j = \sum_i x_i W_{j,i}$$

如果输入和权重量化为：

$$x_i \approx s_x (q_{x,i} - z_x)$$

$$W_{j,i} \approx s_{w,j} (q_{w,j,i} - z_{w,j})$$

那么：

$$y_j \approx s_x s_{w,j} \sum_i (q_{x,i} - z_x) (q_{w,j,i} - z_{w,j})$$

如果是对称量化，通常 $z_x = 0, z_w = 0$ ，就变成：

$$y_j \approx s_x s_{w,j} \sum_i q_{x,i} q_{w,j,i}$$

这时中间的核心计算就从 FP16/BF16 乘加变成 INT8/INT4/FP8 等低精度乘加。

这也是为什么对称量化常用于权重量化：它让矩阵乘更干净。非对称量化虽然表达范围更灵活，但计算时会多出 zero-point 修正项：

$$\sum_i (q_x - z_x)(q_w - z_w) = \sum_i q_x q_w - z_w \sum_i q_x - z_x \sum_i q_w + K z_x z_w$$

这些额外项会增加 kernel 复杂度。

5. 量化省的到底是什么？

主要省三件事。

第一，**省显存**。FP16 一个参数 2 字节，INT8 一个参数 1 字节，INT4 一个参数 0.5 字节。一个 7B 参数模型，FP16 权重大约 14GB；INT4 理论权重约 3.5GB，再加 scale、zero point、packing metadata，实际会略高。

第二，**省显存带宽**。推理时每生成一个 token，都要反复读取权重和 KV cache。很多小 batch 解码场景不是算力不够，而是数据搬运太慢。NVIDIA 的矩阵乘性能指南指出，GEMM 是神经网络里的基本构件；并且矩阵向量乘 GEMV 的 arithmetic intensity 很低，通常是 memory limited。大模型单 token decode 很多时候就接近这种状态。

 NVIDIA Docs

第三，**用上硬件低精度单元**。比如 NVIDIA Tensor Core 是专门做矩阵乘加的硬件单元，执行形式本质上是：

$$D = AB + C$$

早期 Tensor Core 支持 FP16 输入、FP32 累加；后续架构又支持更多低精度格式。NVIDIA 文档说明 Tensor Core 是矩阵乘加单元，并且 CUDA 通过 WMMA 等接口暴露 warp-level matrix operations。  NVIDIA Developer

6. 但量化不一定自动加速

这是很多人容易误解的地方。

量化后模型变小，不等于一定更快。

原因是：如果硬件和算子没有真正支持低比特计算，INT4 权重可能只是“压缩存储”。计算时还要：

```
读取 INT4 packed weight
解包 unpack
反量化 dequantize
转换成 FP16/BF16
再做 GEMM
```



如果这个过程写得不好，反而可能更慢。

所以量化分两层：

```
模型层量化：
    数值变成 INT8 / INT4 / FP8
```



```
算子层适配：
    真的用 INT8 / FP8 Tensor Core 或高效 dequant-GEMM kernel 执行
```

只有这两层都打通，量化才会同时带来显存下降和速度提升。oneDNN 文档也强调，int8 推理需要显式提供 scale、数据类型和输出 scale，而且并不是所有 primitive 都有 int8 版本；有些算子低精度收益很小或精度损失不可接受。  UXL Foundation +1

7. 算子优化的本质是什么？

算子就是模型计算图里的基本计算单元，例如：

```
Linear / GEMM
Attention
Softmax
RMSNorm / LayerNorm
RoPE
SiLU / GeLU
MatMul
Quantize / Dequantize
KV cache read/write
```



算子优化的本质不是改数学公式，而是改执行方式。

比如数学上矩阵乘是：

$$C = AB$$

朴素写法是：

```
for i in M:
    for j in N:
```



```
for k in K:
    C[i,j] += A[i,k] * B[k,j]
```

但 GPU 上不能这么直白地跑。优化后的 GEMM 会做：

- 1. 把 C 切成 tile
- 2. 每个 thread block 负责一个 tile
- 3. 把 A/B 的小块搬到 shared memory
- 4. 在线程寄存器里累加
- 5. 尽量用 Tensor Core 指令
- 6. 最后写回 global memory

NVIDIA 的 GEMM 指南也说明，GPU 实现 GEMM 会把输出矩阵分成 tiles，分配给 thread blocks，并沿 K 维分块加载 A/B、累加输出。  NVIDIA Docs

所以算子优化的核心目标是：

$$T \approx \max \left(\frac{\text{FLOPs}}{\text{Peak Compute}}, \frac{\text{Bytes}}{\text{Memory Bandwidth}} \right) + \text{overhead}$$

也就是：



少搬数据
少 kernel launch

ChatGPT   免費優惠

  ...

让 Tensor Core / SIMD 单元吃满
减少 kernel launch
减少中间结果写回

8. 为什么算子必须适配硬件架构？

因为同一个数学算子，在不同硬件上“最快的写法”完全不同。

以 GPU 为例，它有明显的层级结构：



如果你每算一次都从 HBM 读数据，会非常慢。高性能 kernel 会尽量把数据搬到 shared memory 和 register 里反复使用。CUDA Best Practices 文档说明，shared memory 是片上内存，比 local/global memory 延迟更低、带宽更高，但要避免 bank conflict；也说明 coalesced global memory access 对性能非常重要。  NVIDIA Docs +1

再比如 Tensor Core 对矩阵维度、数据类型、对齐方式都有偏好。NVIDIA GEMM 指南说明，矩阵维度按特定字节倍数对齐时 Tensor Core 效率更好，例如 FP16 常见情况下维度为 8 个元素倍数更高效，A100 上对齐要求/偏好还会更大。  NVIDIA Docs

所以算子不是只写：

```
</> Python
```



▶ 執行

```
C = A @ B
```

而是要考虑：

```
A/B/C 在内存里怎么排布？  
每个 block 算多大 tile？  
每个 warp 做哪一块？  
每个线程负责几个元素？  
shared memory 会不会 bank conflict？  
global memory 访问是否 coalesced？  
寄存器用多了会不会降低 occupancy？  
矩阵维度是否适合 Tensor Core？  
INT4 权重如何 pack/unpack？  
scale 放在哪里？什么时候 dequant？
```



这就是为什么算子需要适配硬件。

9. 算子优化最难的地方

9.1 数学正确性和数值稳定性难

低精度计算会带来：

```
rounding error  
overflow / underflow  
clipping error  
accumulation error  
不同硬件舍入规则不同  
不同计算顺序导致结果不同
```



比如 softmax 不能直接粗暴低精度：

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

如果 x_i 很大，指数会溢出；如果很小，会下溢。所以实际算子要做：

$$\text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}, \quad m = \max_j x_j$$

这类数值稳定技巧会和并行归约、attention tiling、低精度格式耦合起来。

9.2 访存优化比计算优化更难

很多大模型推理瓶颈不是乘加，而是搬数据。尤其 decode 阶段 batch 小、每次生成一个 token，权重和 KV cache 的读取压力极大。

算子优化经常是在解决这个问题：

- 不要把中间结果写回 HBM
- 不要重复读同一块权重
- 不要让线程分散访问内存
- 不要让 shared memory 发生 bank conflict
- 不要让寄存器用太多导致 occupancy 下降



CUDA Best Practices 文档里也提到，shared memory 可以用来减少 global memory 重复传输、改善 coalescing，但 shared memory 和寄存器使用也会影响 occupancy；而 occupancy 并不是越高越好，需要在延迟隐藏和资源使用之间权衡。  NVIDIA Docs +1

9.3 tile size 很难选

以 GEMM 为例，大 tile 复用率高，但并行度可能不够；小 tile 并行度高，但访存复用差、调度开销高。

大 tile：

- 优点：数据复用好，带宽压力小
- 缺点：block 数少，可能吃不满 GPU

小 tile：

- 优点：并行任务多
- 缺点：重复读数据多，效率低



NVIDIA GEMM 指南明确提到，不同 tile size 会带来 tile efficiency 与 tile parallelism 的权衡；大 GEMM 更容易填满 GPU，小 GEMM 则可能因为 tile 效率或并行度不足而无法达到峰值利用率。  NVIDIA Docs

9.4 动态 shape 很难优化

大模型推理的 shape 经常变化：

prompt length 不同
batch size 不同
生成阶段 seq_len 每步增长
KV cache 长度变化
beam search / speculative decoding 改变 batch 结构



一个 kernel 在：

batch=1, seq_len=128



很快，不代表在：

batch=16, seq_len=8192



也快。

所以工程上经常需要很多 kernel：

prefill attention kernel
decode attention kernel
small-batch GEMV kernel
large-batch GEMM kernel
paged KV cache kernel
quantized GEMM kernel
fused RMSNorm kernel



这也是大模型推理框架复杂的根本原因。

9.5 量化和算子是强耦合的

量化参数会直接影响 kernel 设计。

例如 INT4 per-group quantization：

每 128 个权重共享一个 scale
权重 2 个 INT4 pack 到 1 个 uint8
计算时需要 unpack
再乘 scale
再参与矩阵乘



这时 kernel 要同时处理：

低比特解包
scale 加载
dequant
矩阵乘
accumulation
可能还要加 bias / activation / residual

如果 scale 粒度更细，精度更好，但 scale 读取更多，访存更复杂。如果 scale 粒度更粗，kernel 简单，但模型精度可能掉。

所以量化不是纯算法问题，算子也不是纯硬件问题。两者必须一起设计。

10. 最后总结

可以把二者关系理解成：

问题	本质	关注点
大模型量化	数值近似	低比特表示、误差、scale、zero point、outlier
算子优化	执行重排	tile、访存、并行、融合、Tensor Core、寄存器
硬件适配	把数学映射到机器	warp/wavefront、cache、shared memory、指令集、低精度单元

最关键的一句话是：

量化解决“用更少 bit 表示模型还能不能准”；算子优化解决“这些低 bit 数值能不能真的在硬件上跑得快”。

真正高性能的大模型推理系统，必须同时做好：

模型量化算法
+
低比特数据布局
+
硬件友好的 kernel
+
算子融合
+
动态 shape 调度

+
端到端性能 profiling

否则就会出现一种常见情况：**模型看起来已经 INT4 了，但实际推理速度并没有明显提升。**

     ...    来源

$C = A @ B$ 的 @ 是啥？如何系统性学习量化和算子优化

我先把 @ 这个符号讲清楚，再给你一条从数学、PyTorch、量化、CUDA/Triton 到真实推理框架的学习路线。

已思考 30s >

$C = A @ B$ 里的 @ 是 **矩阵乘法运算符**。

在 Python 里，@ 是专门为矩阵乘法引入的运算符；PEP 465 说明它是 matrix multiplication operator，可以理解成“矩阵版的乘号”。NumPy 的 matmul 文档也说明，@ 使用的就是 matmul 语义；PyTorch 的 torch.matmul 也支持向量点积、矩阵乘法、批量矩阵乘法等不同维度情况。  Python Enhance... +2

1. @ 和 * 的区别

假设：

</> Python



▶ 執行

```
import torch
```

```
A = torch.tensor([[1, 2],  
                  [3, 4]])
```

```
B = torch.tensor([[10, 20],  
                  [30, 40]])
```

* 是逐元素乘法

</> Python



▶ 執行

```
A * B
```

结果是：

```
[[1*10, 2*20],  
 [3*30, 4*40]]  
  
=  
[[10, 40],  
 [90, 160]]
```



也就是对应位置相乘。

@ 是矩阵乘法

<> Python



▶ 執行

```
A @ B
```

结果是：

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}$$

也就是：

```
C[0,0] = A[0,0]*B[0,0] + A[0,1]*B[1,0]  
        = 1*10 + 2*30  
        = 70  
  
C[0,1] = 1*20 + 2*40  
        = 100
```



所以：

```
A @ B =  
[[70, 100],  
 [150, 220]]
```



大模型里的线性层，本质上大量都是这种矩阵乘法：

<> Python



▶ 執行

```
Y = X @ W.T
```

其中：

X：输入 token 的隐藏向量

W：线性层权重

Y：输出隐藏向量



2. 大模型为什么到处都是 @

Transformer 里面最核心的计算几乎都可以写成矩阵乘法。

例如注意力：

$$Q = X @ W_q$$
$$K = X @ W_k$$
$$V = X @ W_v$$
$$\text{Attention} = \text{softmax}(Q @ K.T / \sqrt{d}) @ V$$


MLP 层也是：

$$H = \text{activation}(X @ W_1)$$
$$Y = H @ W_2$$


所以大模型推理加速，本质上大量是在加速这些东西：

矩阵乘法

批量矩阵乘法

低精度矩阵乘法

带反量化的矩阵乘法

注意力算子

归一化算子

激活函数算子

KV cache 读写



3. 如何系统性学习量化和算子优化

你可以按下面这条路线学，不要一上来就看 CUDA kernel，否则很容易被细节淹没。

第一阶段：先把矩阵乘法和张量维度吃透

你先要非常熟悉：

向量点积
矩阵乘法
批量矩阵乘法
张量 broadcasting
GEMV
GEMM



重点不是公式，而是 shape。

例如：

</> Python



▶ 執行

```
X.shape = [batch, seq_len, hidden]
W.shape = [hidden, hidden]
Y = X @ W
Y.shape = [batch, seq_len, hidden]
```

这里的 @ 会对最后两个维度做矩阵乘法，前面的维度当成 batch 维度。

你要能看到一个 Transformer 层，马上判断：

输入 shape 是什么？
权重 shape 是什么？
输出 shape 是什么？
这个算子是 memory-bound 还是 compute-bound？



第二阶段：学习量化的数学本质

量化先学这几个概念：

scale
zero point
symmetric quantization
asymmetric quantization
per-tensor quantization
per-channel quantization
per-group quantization
weight-only quantization
activation quantization
KV cache quantization



最核心公式是：

$$q = \text{round}(x/s)$$

$$\hat{x} = sq$$

也就是：

原始 FP16/FP32 数值 x

↓

除以 $scale$

↓

round 到整数

↓

存成 INT8 / INT4

↓

计算时再乘 $scale$ 近似恢复



更完整的 affine quantization 是：

$$q = \text{round}(x/s + z)$$

$$\hat{x} = s(q - z)$$

PyTorch torchao 文档说明，大多数张量量化使用 affine quantization，也就是通过 $scale$ 和 $zero\ point$ 把高精度张量映射到低精度张量。 [PyTorch Docs](#)

你学习量化时，要一直记住一个目标：

量化不是为了让每个权重都完全一样，而是为了让模型输出尽量不变。

数学目标可以理解成：

$$f_{\theta}(x) \approx f_{Q(\theta)}(x)$$

其中 $Q(\theta)$ 是量化后的参数。

第三阶段：亲手实现一个最小量化 Linear

不要先看 GPTQ、AWQ、SmoothQuant。先自己写一个最小版本。

目标：

Python



▶ 執行

原始：

$Y = X @ W.T$

量化：

```
W_int8 = round(W / scale)
W_hat = W_int8 * scale
Y_hat = X @ W_hat.T
```

你要比较三件事：

1. W 和 W_{hat} 的误差
2. Y 和 Y_{hat} 的误差
3. 端到端推理速度有没有变快



你会马上发现一个重要事实：

权重量化了，不代表一定加速。

因为如果你只是：

Python



▶ 執行

```
W_hat = W_int8.float() * scale
Y = X @ W_hat.T
```

那你只是“压缩存储”，计算时又反量化回 FP16/FP32 了。

真正的加速需要低精度 kernel 支持。

这就是量化和算子优化连接起来的地方。

第四阶段：学习主流 LLM 量化方法

可以按这个顺序学：

1. FP16 / BF16 mixed precision
2. INT8 weight-only quantization
3. INT8 weight + activation quantization
4. INT4 weight-only quantization
5. GPTQ
6. AWQ
7. SmoothQuant
8. FP8 / MXFP8 / NVFP4
9. KV cache quantization



你不要一开始纠结所有论文细节。先抓住它们分别在解决什么问题：

方法	核心问题
INT8	降低权重和激活精度
INT4	进一步压缩权重
GPTQ	用二阶近似降低权重量化误差
AWQ	保护重要通道，减少量化损伤
SmoothQuant	把 activation outlier 平滑转移到 weight
FP8	用硬件友好的浮点低精度格式
KV cache quantization	降低长上下文推理显存占用

vLLM 文档也明确说，量化是在模型精度和更小内存占用之间做权衡，并列出了 FP8、INT8、INT4、GPTQ、AWQ、GGUF、TorchAO 等多种部署格式。  vLLM +1

第五阶段：学习算子优化的本质

算子优化不是换公式，而是换执行方式。

比如数学上都是：

$$C = A@B$$

但是朴素实现是：

 Python



 執行

```
for i in range(M):
    for j in range(N):
        for k in range(K):
            C[i, j] += A[i, k] * B[k, j]
```

高性能 GPU 实现会变成：

- 1. 把 C 切成很多 tile
- 2. 每个 CUDA block 负责一个 tile
- 3. 把 A/B 的小块搬到 shared memory
- 4. 在线程寄存器里累加
- 5. 尽量使用 Tensor Core
- 6. 最后写回 global memory



NVIDIA CUTLASS 文档把这些底层概念暴露出来，包括 layouts、tensors、hardware atoms，以及线程和数据层级控制；Triton 的官方矩阵乘教程也专门教如何写一个可扩展的高性能 matmul kernel。  NVIDIA Docs +1

第六阶段：建立性能分析思维

算子优化一定要学这个公式：

$$T \approx \max \left(\frac{\text{计算量}}{\text{峰值算力}}, \frac{\text{访存量}}{\text{内存带宽}} \right)$$

意思是：

如果计算很多、数据复用好：

compute-bound，瓶颈是算力

如果计算不多、搬数据很多：

memory-bound，瓶颈是显存带宽



大模型 decode 阶段经常是 memory-bound，因为每生成一个 token，都要读大量权重和 KV cache。

所以算子优化不是盲目“多线程”，而是分析：

到底是算得慢？

还是搬数据慢？

还是 kernel launch 多？

还是数据 layout 不合适？

还是反量化开销太大？



第七阶段：先学 Triton，再学 CUDA / CUTLASS

对你来说，我建议顺序是：

PyTorch

↓

torch.compile / torch profiler

↓

Triton

↓

CUDA 基础

↓

CUTLASS



↓
TensorRT-LLM / vLLM / llama.cpp

原因是：

PyTorch：方便理解模型计算图
Triton：比较容易写自定义 GPU kernel
CUDA：理解硬件底层
CUTLASS：学习工业级 GEMM 模板
TensorRT-LLM / vLLM：学习真实部署系统
llama.cpp：学习 CPU / 本地推理 / GGUF 量化



TensorRT 的官方介绍也提到，它通过量化、layer/tensor fusion 和 kernel tuning 来优化推理；TensorRT-LLM 则提供 LLM 部署和量化相关文档。  NVIDIA Developer +1

4. 推荐你的实战项目路线

你可以按下面 6 个小项目学。

项目 1：实现矩阵乘法

用 Python 写：

Python



▶ 執行

```
C = A @ B
```

然后自己用三层 for 循环实现一遍：

Python



▶ 執行

```
for i in range(M):  
    for j in range(N):  
        for k in range(K):  
            C[i, j] += A[i, k] * B[k, j]
```

目标是理解：

M、N、K 是什么
GEMM 是什么
为什么矩阵乘法是大模型核心



项目 2：实现 INT8 权重量化 Linear

做：

FP16 Linear
INT8 weight-only Linear
比较误差
比较显存
比较速度



你会理解：

scale 怎么选
per-tensor 和 per-channel 有什么区别
量化误差怎么传播到输出



项目 3：实现 INT4 group-wise 量化

做：

每 128 个权重一组
每组一个 scale
权重打包成 INT4
计算时 unpack + dequant



这个项目非常关键，因为它会让你理解：

INT4 真正难的不是公式，而是数据 layout、unpack、scale 加载和 matmul 的融合。

项目 4：用 Triton 写一个 matmul kernel

先写 FP16 matmul。

目标不是超过 cuBLAS，而是理解：

tile
block
program id
shared memory / SRAM
register
BLOCK_M / BLOCK_N / BLOCK_K
num_warps



Triton 官方矩阵乘教程就是从这个角度教你实现高性能 matmul。  Triton

项目 5：写 fused dequant + matmul

这是量化和算子优化真正结合的地方。

目标：

输入：X_fp16, W_int4, scale

输出：Y_fp16

不要先完整反量化 W

而是在 matmul kernel 内部边 unpack、边 dequant、边乘加



也就是把：

unpack

dequant

matmul



融合成一个 kernel。

这就是很多低比特推理 kernel 的核心思想。

项目 6：阅读真实推理框架

最后再看：

llama.cpp

vLLM

TensorRT-LLM

CUTLASS

torchao



看这些项目时，不要试图一口气读完整代码，而是带着问题读：

它的权重格式是什么？

scale 存在哪里？

INT4 怎么 pack？

反量化在哪里做？

matmul 调的是哪个 kernel？

prefill 和 decode 是否用不同 kernel？

KV cache 怎么管理？

是否支持 tensor parallel？



llama.cpp 的量化工具说明，它会把 F32/BF16 等高精度 GGUF 模型转换成低比特格式，以减小模型大小并可能提升推理速度，但会引入通常用 perplexity 或 KL divergence 衡量的精度损失。  [GitHub](#)

5. 你应该怎么安排学习顺序

我建议你用这个 8 周路线。

周数	学习内容	产出
第 1 周	矩阵乘法、张量 shape、GEMM/GEMV	手写 matmul + shape 笔记
第 2 周	PyTorch Linear、Attention、MLP	拆解一个小 Transformer
第 3 周	INT8 量化数学	写 INT8 Linear
第 4 周	INT4 group quantization	写 INT4 权重打包/解包
第 5 周	PyTorch profiler、显存/带宽分析	benchmark FP16 vs INT8/INT4
第 6 周	Triton matmul	写 FP16 matmul kernel
第 7 周	fused dequant-matmul	写 INT4 dequant + matmul
第 8 周	vLLM / llama.cpp / TensorRT-LLM	写一份推理框架阅读报告

6. 最重要的学习抓手

你要始终围绕这条主线：

数学上：

$$Y = X @ W.T$$

量化后：

$$W \approx scale * W_{int}$$

工程上：

不要先 `W_int -> W_fp16` 再 `matmul`
而是把 `dequant` 和 `matmul` 融合

硬件上：

数据必须按 `Tensor Core / SIMD / cache / memory coalescing` 的方式排列

一句话总结：

量化学习的核心是误差控制；算子优化学习的核心是数据搬运和硬件并行；二者结合的核心是 fused low-bit matmul。

对你来说，最值得做的第一个完整项目是：

从零实现一个小型 INT4 weight-only Linear：

1. FP16 baseline
2. per-group INT4 quantization
3. 误差评估
4. PyTorch 反量化版本
5. Triton fused dequant-matmul 版本
6. 对比显存、速度、误差



这个项目做完，你对“大模型量化 + 算子优化”的理解会比只看十篇论文扎实得多。

     ...    来源

把你全部的回答，完整保存成md文档，包括公式

下面是整理好的 Markdown 文档，你可以直接保存为：

LLM_Quantization_And_Operator_Optimization.md



大模型量化与算子优化：本质、数学机理、硬件适配与学习路线

目录

1. 大模型量化的本质
2. 量化的数学机理
3. 为什么大模型量化困难
4. 量化如何降低算力和显存
5. 量化真正节省了什么
6. 为什么量化不一定加速
7. 算子优化的本质
8. 为什么算子必须适配硬件
9. 算子优化的难点